

Интернет

Стек протоколов

TCP/IP Stack

1. Physical
2. Data link
3. Network
4. Transport
5. Application

OSI model

1. Physical
2. Data link
3. Network
4. Transport
5. Session
6. Presentation
7. Application

Physical Layer

Железо получает-отправляет биты по проводу или воздуху

Data Link Layer

Ethernet, Wi-Fi(IEEE 802.11)

```
struct DataLinkDest { mac } // in LAN

interface DataLinkProtocol {
    recieveFrame() -> DataLinkDest, bytes
    sendFrame(DataLinkDest, bytes)
}
```

Data Link Layer

ARP (address resolution protocol)

```
interface ArpProtocol {  
    getMac(ip) -> mac // ip in LAN  
}
```

Network Layer

```
struct NetworkDest { ip } // in global network
```

IP

```
struct IpHeader {  
    sourceIp  
    destIp  
    ...  
}  
  
interface IpProtocol {  
    recievePacket() -> NetworkDest, bytes  
    sendPacket(dst: NetworkDest, bytes) {  
        frame = bytes ++ IpHeader{}  
        ipToSend = routingTable.next(dst.ip)  
        mac = ArpProtocol.getMac(ipToSend)  
        DataLinkProtocol.sendFrame(mac, frame)  
    }  
}
```

Transport Layer

```
struct TransportDest { ip, port }
```

TCP

```
interface TcpConnection {  
    close()  
    stream: FullDuplexStream  
}  
  
interface FullDuplexStream {  
    read() -> bytes  
    write(bytes)  
}  
  
interface TcpProtocol {  
    connect(TransportDest) -> TcpConnection  
}
```

Transport Layer

TCP

```
struct TcpHeader {
    srcPort,
    tcpPort,
    sequenceNumber,
    type // (ACK / FIN / etc) for special packets
}

interface FullDuplexStream {
    dest: TransportDest

    read() -> bytes
    write(bytes) {
        packet = bytes ++ TcpHeader{}
        IpProtocol.sendPacket(dst.ip, packet)
        // await ack or retry
    }
}
```


Transport Layer

```
struct TransportDest { ip, port }
```

UDP

```
interface UdpProtocol {  
    recieveDatagram() -> TransportDest, bytes  
    sendDatagram(TransportDest, bytes)  
}
```

Application Layer

```
struct ApplicationDest { ip, port }
```

HTTP

```
struct HttpRequest { path, headers, body }

struct HttpResponse { status, headers, body }

interface HttpProtocol {
    get(HttpRequest) HttpResponse
    post(HttpRequest) HttpResponse
    put(HttpRequest) HttpResponse
    delete(HttpRequest) HttpResponse
    ...
}
```